

Prove a mano dei due banchi — ramo 1

*16 settembre 2026. Le cinque prove del ramo 1 di prompt-prossima-sessione-giano.md, eseguite sul simulatore del Dev Kit con il banco WinForms e con il banco Compose. **Tutte e cinque superate in tutti e due i banchi**, con gli stessi messaggi. Nessuna richiedeva la macchina vera.*

Punto 6 del ramo 1: fatto. `cancelCurrent()`, che nel banco Compose non chiamava più nessuno, è stata tolta. Il [AN] usa `cancelOperation()`, come il [AN] del WinForms usa `CancelAsync()`: a incasso aperto passano per la via laterale e restituiscono **l'esito dell'incasso**.

Com'è stata condotta la sessione

Simulatore del Dev Kit su 127.0.0.1:9100, giacenze riempite con Fill (monete 57,50, banconote 620,00), terminatore \r, **diagnostica libreria** e **registrazione su file** accese in tutti e due i banchi. I log sono winforms-2026-09-16.log e compose-2026-09-16.log in %LOCALAPPDATA%\PayPrint.PagAmico\logs.

Il cliente virtuale del simulatore inserisce circa 200 € al secondo. È la cosa da sapere per ripetere queste prove: con importi piccoli l'incasso **si chiude da solo** prima che si riesca a premere un pulsante, e i click finiscono su un incasso già chiuso (è successo due volte, ed è il motivo per cui i primi tentativi non provavano quello che dovevano). Con **3.000 €** la finestra è di una quindicina di secondi, con **9.000 €** di circa quarantacinque: sono gli importi usati qui. Il tetto del protocollo è 9.999,99 €. Nel collaudo automatico lo stesso problema è gestito diversamente: [IN]+[CM] aspetta 2,5 s e, se l'incasso è già chiuso, lo dice e suggerisce di alzare *«Ritardo fra un pezzo e l'altro»* nel simulatore.

Il criterio comune a tutte le prove sono le righe tx: il banco ne scrive una per ogni comando che parte davvero, quindi «nulla trasmesso» si legge lì.

1. Incasso, poi [AN] a incasso aperto — superata

Incasso da 3.000 € (WinForms) e da 9.000 € (Compose), [AN] premuto con 800 € già inseriti.

```
WinForms 09:20:23.866 i [AN] annullo
          09:20:23.882 i attesa di 'IN300000' soddisfatta da: {"response":"AN",...}
          09:20:23.885 TX > AN
          09:20:23.887 + response=AN incassato=800,00 restituito=800,00 Id=7245 error
Type=OK
          09:20:23.923 + response=AN incassato=800,00 restituito=800,00 Id=7245 error
Type=OK
Compose 09:28:40.118 + response=AN incassato=800.0 restituito=800.0 Id=7246 errorTy
pe=OK
```

L'esito dell'incasso compare, con gli importi, e il denaro è stato restituito. Un solo TX > AN.

Le due righe di riepilogo identiche sono corrette: una la scrive l'attesa dell'[IN], l'altra la chiamata di [AN], che dall'11 settembre restituisce lo stesso esito. Non è un doppio invio.

2. Incasso, poi [CM] — superata

```
09:20:48.279 RX < {"response":"CM",..., "errorCode":"OK", "committedAmount":0.0} <- acce
ttazione
09:20:48.287 TX > CM
09:20:48.799 RX < {"response":"CM",..., "errorCode":"", "committedAmount":800.0} <- esit
o
09:20:48.844 + response=CM incassato=800,00 trattenuto=800,00 (controllo committedAmoun
t=800,00)
```

D1 non è tornato: la libreria chiude sul secondo frame, quello con errorCode vuoto, e legge 800,00 dove prima avrebbe letto 0,00. collectedAmount e committedAmount coincidono, quindi la diagnostica di controllo non segnala niente. Identico in Compose (trattenuto=800.0).

Confermata anche la forma dei due frame descritta nell'esito: **ciò che li distingue è errorCode** (OK nell'accettazione, vuoto nell'esito). Nel JSON del simulatore il campo si chiama committedAmount, senza la *n*: è un refuso del protocollo, e la libreria lo mappa già così. Nessun CM in ritardo fra gli orfani in questa sessione.

3. Seconda chiusura sullo stesso incasso — superata

[CM] e subito dopo [AN], con il commit ancora in volo:

```
WinForms 09:21:19.726 TX > CM
          09:21:20.073 i [AN] annullo
          09:21:20.091 ERR! Chiusura dell'incasso gia' richiesta con CM: 'AN' non inviato
Compose 09:29:18.262 ERR! Chiusura dell'incasso gia' richiesta con CM: 'AN' non inviato
```

Nessun TX > AN: il secondo comando non è stato trasmesso. Il CM è poi andato a buon fine.

4. Un altro comando a incasso aperto — superata

[ST] Richiesta situazione premuto con un incasso da 9.000 € aperto:

```
09:22:22.891 i [ST] situazione
09:22:22.894 i 'ST' NON inviato: incasso aperto, la macchina accetta solo AN e CM
09:22:22.920 ERR! Incasso aperto: 'ST' non inviato, durante l'incasso la macchina accetta so
lo AN e CM
```

Nessun TX > ST, incasso rimasto aperto e chiuso dopo con [AN] (denaro restituito). Identico in Compose. Il blocco degli invii laterali regge anche premendo i pulsanti a mano.

5. Frame orfani nel log — superata

Sessione di ricarica mista (vs) con l'opzione *invia i parziali al client*:

```
09:23:25.424 TX > VS
09:23:26.223 i    messaggio non atteso da nessuno (nessun comando in corso): {"response":"p",...}
09:23:26.224 ERR! frame orfano (nessuna attesa lo riconosce): {"response":"p","collectedAmount":1002.0,...}
```

Tre frame orfani per banco, **con gli importi dentro** (collectedAmount che sale 1002, 1004, 1006). È esattamente il canale su cui in Giano andranno salvati gli importi che oggi si perderebbero: Decisione 2 di decisioni-innesto-giano.md.

Tre cose emerse — due corrette lo stesso giorno

- **La riga TX > compariva dopo la risposta. Corretta.** L'evento `CommandSent / onCommandSent` era sollevato *dopo* la scrittura sul socket, e su 127.0.0.1 la risposta a volte veniva letta prima: nel log del [CM] la riga RX era a .279 e il TX > CM a .287. Era l'ordine delle righe, non dei byte, ma un log con dentro un incasso diventava illeggibile — e con il Tap in mezzo lo sarebbe stato di più. La notifica è ora chiamata sotto il lock di scrittura, subito prima dei byte, **nelle due librerie** e anche per i pacchetti immagine; un test offline per parte lo fissa (171 in C#, 172 in Kotlin) e i due collaudi restano 64 su 64. Nel log del collaudo del 16 settembre il TX ST precede ora la lettura dal socket.
- **I due banchi formattavano gli importi in modo diverso. Uniformati.** WinForms 800,00 (formato 0.00), Compose 800.0 (il `toString` di `BigDecimal`): due log della stessa prova non si potevano confrontare. Il banco Compose ha ora l'helper `BigDecimal?.eur()` con lo stesso formato, usato nel riepilogo e nei parziali; verificato sul simulatore (parziale: incassato 1,00 (monete 1,00, banconote 0,00), da incassare 0,50).
- **Un incasso rifiutato si riconosce.** Un tentativo con importo malformato è partito come `IN000000` e la macchina ha risposto `CMD ERROR: la libreria ha chiuso con Incasso 'IN000000' rifiutato: CMD ERROR`, cioè `PagAmicoRejectedException`. È il predicato in due fasi del punto 2 delle correzioni dell'11 settembre, visto sul simulatore vero: prima dell'ok un testo significa incasso rifiutato.

Stato lasciato al simulatore

Le prove hanno trattenuto 1.400 € con i due CM (800 + 600) e incassato 3.000 € con un incasso completato per intero; tutti gli [AN] hanno restituito il denaro. Il simulatore segnala ora `Monete sottoscorta` e `Troppe banconote`: eseguire scarico banconote: per rimetterlo in ordine basta `Fill (monete)` o uno scarico banconote dal pannello.

Che cosa resta del ramo 1

Niente: i punti 1-6 sono chiusi, e le due correzioni nate da questa sessione sono fatte e provate. **Non** sono state toccate le cose che vanno dopo le prove sulla macchina vera: default del terminatore, pausa di 80 ms, timeout di 5 minuti (D2), chiusura dell'incasso su ER dopo l'ok.